

Project2-基于深度学习的调制识别

(一) 实验目标:

- 1、了解深度学习在通信电子战中的应用
- 2、了解卷积神经网络 CNN
- 3、能够利用训练好的 CNN 网络进行调制分类
- 4、能够利用 MATLAB 的深度学习工具箱训练 CNN 网络
- 5、了解 JetsonNano 上部署 CNN，并实现实时调制分类

(二) 实验安排:

第一部分：深度学习与通信电子战

1、实验条件：MATLAB (2022b)、MATLAB 通信工具箱、MATLAB 数字信号处理工具箱、深度学习工具箱

- 2、课堂讲解：无线通信干扰识别
- 3、课堂讲解：认知电子战中的调制识别问题
- 4、课堂练习：Modulation Classification with Deep Learning

(1) 打开实时脚本（注意需要安装通信工具箱和深度学习工具箱）

(2) 在程序中选择 Use trained network

(3) 随机设置调制方式，观察基于 CNN 网络的识别概率。

5、编程作业:

(1) 核对是否已经完成深度学习工具箱（Deep Learning Toolbox）安装

(2) 设调制方式分别为 BPSK、16-QAM、64-QAM、GFSK、CPFSK、B-FM、DSB-AM、SSB-AM，画出对应的识别概率图，并且在 Blackboard 系统课堂实时提交。

(3) 步入函数: helperModClassGetNNFrames、classify, 根据 MATLAB 给出的说明, 了解函数功能。

第二部分: 卷积神经网络

1、实验条件: MATLAB (2022b)、MATLAB 通信工具箱、MATLAB 数字信号处理工具箱、深度学习工具箱。

2、课堂讲解: CNN 调制分类流程图

3、课堂讲解: 调制实现参考 MATLAB 中的调制函数

数字调制: BPSK、QPSK、8-PSK、16-QAM、64-QAM、4-PAM、GFSK、CPFSK

模拟调制有: B-FM、DSB-AM、SSB-AM

4、课堂讲解: 无线通信基础, 复基带波形、脉冲成形、高斯白噪声、信道时延、多径效应、多普勒频移、数据帧

5、课堂讲解: 深度学习工具箱中的分类函数 classify, 利用 CNN 网络对输入数据进行分类, 返回预测的分类, 以及对应的得分。

6、课堂讲解: 深度学习的基本概念, 图像特征、卷积核、CNN 卷积过程、非线性激活函数、最大池化

7、课堂讲解: 多层网络 and 全连接层, 对特征图进行维度上的改变, 来得到每个分类类别对应的概率值。

8、课后作业:

(1) 画出以下函数的流程图:

helperModClassFrameGenerator、helperModClassFrameGenerator

2、画出以下函数的流程图：classify

第三部分：训练卷积神经网络

1、实验条件：MATLAB（2022b）、MATLAB 通信工具箱、MATLAB 数字信号处理工具箱、深度学习工具箱。

2、课堂讲解：训练设置，产生 400 个帧数据，80%用于训练，10%用于评估网络，10%用于测试。开始用训练和评估帧，最后分类测试用测试帧。

3、课堂讲解：训练卷积神经网络的，显示各种调制对应时域波形和频域图

4、课堂讲解：将复信号变换为实数数组、训练集和测试集比例划分、向内存中导入数据、训练 CNN、分类测试、混淆矩阵。

附录 1：程序注释

本例中，使用卷积神经网络 (CNN) 进行调制识别，包括 8 种数字调制方式，3 种模拟调制方式：

数字调制有：BPSK、QPSK、8-PSK、16-QAM、64-QAM、4-PAM、GFSK、CPFSK

模拟调制有：B-FM、DSB-AM、SSB-AM

1、定义需要分类的调制类型

```
modulationTypes = categorical(["BPSK", "QPSK", "8PSK", "16QAM", "64QAM",  
"PAM4", "GFSK", "CPFSK", "B-FM", "DSB-AM", "SSB-AM"]);
```

2、加载一个已经训练好的网络。

```
load trainedModulationClassificationNetwork  
trainedNet  
trainedNet =  
    SeriesNetwork - 属性：  
        Layers: [28x1 nnet.cnn.layer.Layer]  
        InputNames: {'Input Layer'}  
        OutputNames: {'Output'}
```

每一帧包含 1024 个采样值。无线信号经过莱斯多径衰落、频偏、延时和加性高斯白噪声信道。

需要使用到的函数：

- [randi](#): 产生随机比特
- [pammod](#): PAM4-调制比特
- [rcosdesign](#): 设计一个根升余弦滚降滤波器
- [filter](#): 脉冲成形滤波器
- [comm.RicianChannel](#): Rician 多径信道
- [comm.PhaseFrequencyOffset](#): 由于时钟偏移导致的频偏和相偏
- [interp1](#): 由于时钟偏移引起的同步
- [awgn](#): 添加 AWGN 信道

% 2.1 设置随机种子，保证每次产生的随机比特一致。

```
%rng(123456)
```

% % 2.2 产生随机符号，这里产生 0, 1, 2, 3 四个数字，每个数字对应一个调制符号，1024 个调制符号，

```
% d = randi([0 3], 512, 1);
```

```
%
```

% % 2.3 调制，注意这里的映射关系

```
% syms = qammod(d,4);
```

```
%
```

% % 2.4 根升余弦滚降滤波器，注意这里滚降因子等于 0.35，符号周期为 4，采样率为 8

```
% filterCoeffs = rcosdesign(0.35,4,8);
```

```
%
```

% % 2.5 上采样，获得复基带波形

```
% tx = filter(filterCoeffs,1,upsample(syms,8));
```

% % 2.6 DSB-AM 调制

```
% tx = AMmodulation(spf,fs);
```

% % 2.7 FM 宽带调频

```
    spf = 1024;           % 每帧 1024 个采样
```

```
    fs = 200e3;
```

```
    tx = FMmodulation(2*spf,fs);
```

% 3.0 设置信道参数

% 3.1 信噪比

```
SNR = 30;
```

% 3.2 最大偏移

```
maxOffset = 5;
```

% 3.3 fc 表示载波频率 902MHz

```
fc = 902e6;
```

%fs 表示采样率 200kps

```
fs = 200e3;
```

```

%多径信道由通信系统工具箱给出，莱斯信道
multipathChannel = comm.RicianChannel('SampleRate', fs, 'PathDelays', [0
1.8 3.4] / 200e3, 'AveragePathGains', [0 -2 -10], 'KFactor', 4,
'MaximumDopplerShift', 4);

% 3.4 设置频偏
frequencyShifter = comm.PhaseFrequencyOffset('SampleRate', fs);

% 3.5 应用多径信道，获得输出波形
reset(multipathChannel)
outMultipathChan = multipathChannel(tx);

% 3.6 计算时钟偏移因子
clockOffset = (rand() * 2*maxOffset) - maxOffset;
C = 1 + clockOffset / 1e6;

% 3.7 增加频偏，获得频偏后的信号
frequencyShifter.FrequencyOffset = -(C-1)*fc;
outFreqShifter = frequencyShifter(outMultipathChan);

% 3.8 增加随机延时
t = (0:length(tx)-1)' / fs;
newFs = fs * C;
tp = (0:length(tx)-1)' / newFs;
outTimeDrift = interp1(t, outFreqShifter, tp);

% 3.9 增加噪声
rx = awgn(outTimeDrift,SNR,0);

% 3.10 数据帧生成，要利用该模型，需要产生一定格式的数据帧
unknownFrames = helperModClassGetNNFrames(rx);

% 3.11 分类
[prediction1,score1] = classify(trainedNet,unknownFrames);

```

返回预测类型

prediction1

返回各种分类得分情况。

```
helperModClassPlotScores(score1,modulationTypes)
```

在使用 CNN 进行调制分类之前，我们需要利用已知的数据去训练 CNN 网络。接下来内容分为三个部分：第一部分介绍通信工具箱中的函数模块如，调制、滤波器、信道，产生训练数据。第二部分介绍定义、训练和 CNN 测试；第三部分介绍利用软件无线电平台进行调制分类

1、波形训练

对于各种调制类型，产生 400 个帧数据，80%用于训练，10%用于确认，10%用于测试。用训练和确认数据帧进行训练。最后分类精度用测试帧进行提升。

每个帧长度是 1024 个采样，采样率为 200kHz

对于数字调制，8 个采样表示一个符号。

假设数字信号的中心频率是 902MHz，模拟信号的中心频率是 100MHz

为了获得 CNN 网络，选择"Train network now"

```
trainNow = true;
if trainNow == true
    numFramesPerModType = 400;
else
    numFramesPerModType = 200;
end

percentTrainingSamples = 80;    % 训练集：训练模型
percentValidationSamples = 10;  % 验证集：评估模型
percentTestSamples = 10;       % 测试集：一旦找到了最佳参数，就开始最终训练
sps = 8;                       % 每个符号 8 个采样
spf = 1024;                    % 每帧 1024 个采样
symbolsPerFrame = spf / sps;   % 每帧 128 个采样
fs = 200e3;                    % 采样率为 200kHz
fc = [902e6 100e6];            % 中心频率，数字调制 902MHz，模拟调制 100MHz
```

2、创建信道

每个数据帧经过独立信道，依次通过 AWGN，莱斯多径、时钟偏移（导致的频偏和时偏）。

AWGN 中，设信道的信噪比 SNR 为 30dB

莱斯多径衰落模型中，延时模型[0 1.8 3.4] 个采样，对应的路径衰减为[0 -2 -10] dB，K-因子为 4，最大多普勒频移是 4Hz

时钟偏移模型，因为不准确的内部时钟源。时钟偏移会导致本征频率漂移，从而导致数模转换器采样值偏离理想值。

信道仿真器用时钟漂移因子, $C = 1 + \frac{\Delta_{\text{clock}}}{10^6}$ ， Δ_{clock} 是时钟偏移。对于每一帧，信道产生一个随机时钟偏移，该值服从均匀分布。时钟偏移单位是 ppm，设最大时钟偏移为 5 ppm。

```
maxDeltaOff = 5;
deltaOff = (rand()*2*maxDeltaOff) - maxDeltaOff;
C = 1 + (deltaOff/1e6);
```

3、组合信道

使用函数 helperModClassTestChannel 可以将三种信道 设置一个数据帧。

```
channel = helperModClassTestChannel(...
    'SampleRate', fs, ...
    'SNR', SNR, ...
    'PathDelays', [0 1.8 3.4] / fs, ...
    'AveragePathGains', [0 -2 -10], ...
    'KFactor', 4, ...
    'MaximumDopplerShift', 4, ...
    'MaximumClockOffset', 5, ...
    'CenterFrequency', 902e6)
```


查看信道信息

```
chInfo = info(channel)
```

4、波形产生

创建一个循环，产生信道后的数据帧，并且保存相应的调制类型标签

```
% 设置种子
rng(1235)

%记录执行时间
tic

%所有的分类数目
numModulationTypes = length(modulationTypes);

%信道信息
channelInfo = info(channel);

%传播延时
transDelay = 50;

%数据文件夹
dataDirectory = fullfile(tempdir, "ModClassDataFiles");
disp("Data file directory is " + dataDirectory)
fileNameRoot = "frame";

% Check if data files exist
dataFilesExist = false;
if exist(dataDirectory, 'dir')
    files = dir(fullfile(dataDirectory, sprintf("%s*", fileNameRoot)));
    if length(files) == numModulationTypes*numFramesPerModType
        dataFilesExist = true;
    end
end
```

```

if ~dataFilesExist
    disp("Generating data and saving in data files...")
    [success,msg,msgID] = mkdir(dataDirectory); % 相同的文件夹创建 2 次, msg
提示: 返回目录已经存在
    if ~success
        error(msgID,msg)
    end

%从第一种开始
for modType = 1:numModulationTypes
    elapsedTime = seconds(toc);
    elapsedTime.Format = 'hh:mm:ss';

%产生 BPSK 帧
    fprintf('%s - Generating %s frames\n', ...
        elapsedTime, modulationTypes(modType))

%调制类型
    label = modulationTypes(modType);
    % numSymbols = (numFramesPerModType / sps);
    % 返回调制类型对应的比特产生程序
    dataSrc = helperModClassGetSource(modulationTypes(modType), sps, 2*spf,
fs);

    % 返回调制类型对应的调制函数, 这个处理中包含调制、上采样、脉冲成形
    modulator = helperModClassGetModulator(modulationTypes(modType), sps,
fs);

    % 如果包含字符 B-FM, DSB-AM, SSB, 设置频率为 100M, 否则设置为 902M
    if contains(char(modulationTypes(modType)),
{'B-FM', 'DSB-AM', 'SSB-AM'})

        channel.CenterFrequency = 100e6;
    else

        channel.CenterFrequency = 902e6;
    end

    % 循环开始, 每种调制 400 个调制帧
    for p=1:numFramesPerModType

```

```

% 随机产生数据比特
x = dataSrc(); % 产生调制方式对应的符号，产生 256 个符号，2048 个采样

% 调制，上采样和脉冲成形，得到的是复基带信号
y = modulator(x);

% 通过组合衰落信道，模拟接收机接收到的信号
rxSamples = channel(y);

% 从复基带波形中选择 1024 个采样点作为一帧
% transDelay = 50，起始值索引是 50+4, 50 是传输延时，4 是 0-8 产生的随机数，
frame 是 1024X1，归一化
frame = helperModClassFrameGenerator(rxSamples, spf, spf, transDelay,
sps);

% 保存文件
fileName = fullfile(dataDirectory,...
    sprintf("%s%s%03d",fileNameRoot,modulationTypes(modType),p));
save(fileName,"frame","label")
end
end

else
    disp("Data files exist. Skip data generation.")
end
end

```

```

% 随机选择一帧，显示同相分量和正交分量
helperModClassPlotTimeDomain(dataDirectory,modulationTypes,fs)

```

```

% 画出对应的频谱
helperModClassPlotSpectrogram(dataDirectory,modulationTypes,fs,sps)

```

```

frameDS =
signalDatastore(dataDirectory,'SignalVariableNames',['frame','label']);

```

% 将复信号变换为实数数组

% 此示例中的深度学习网络需要实数输入，而接收的信号包含复数基带采样。将复信号变换为实数值四维数组。

% 输出帧的大小为 $1 \times \text{spf} \times 2 \times N$ ，其中第一页（第三个维度）是同相采样，第二页是正交采样。

% 当卷积滤波器的大小为 $1 \times \text{spf}$ 时，这种方法可确保混合 I 和 Q（甚至是卷积层）中的信息，使我们能够更好地利用相位信息。

```
frameDSTrans = transform(frameDS,@helperModClassIQAsPages);
```

% 按照百分比划分，训练集 80%，模型评估集占 10%，测试集占 10%

```
splitPercentages =
[percentTrainingSamples,percentValidationSamples,percentTestSamples];
[trainDSTrans,validDSTrans,testDSTrans] =
helperModClassSplitData(frameDSTrans,splitPercentages);
```

%神经网络训练是迭代进行的。在每次迭代中，数据存储从文件中读取数据，变换数据，然后更新网络系数。如果数据可放入计算机的内存中，则将数据从文件导入内存可以消除重复的文件读取和变换过程，从而加快训练速度。这样，只需执行一次从文件读取并变换数据的操作。使用磁盘上的数据文件训练此网络需要大约 110 分钟，而使用内存中的数据只需要大约 50 分钟。

% 将文件中的所有数据导入内存。这些文件有两个变量：**frame** 和 **label**，对数据存储的每个 [read](#) 调用都返回一个元胞数组，其中第一个元素是 **frame**，第二个元素是 **label**。

使用 [transform](#) 函数：

[helperModClassReadFrame](#) 和 [helperModClassReadLabel](#) 读取帧和标签。

```
% Read the training and validation frames into the memory
pctExists = parallelComputingLicenseExists();
trainFrames = transform(trainDSTrans, @helperModClassReadFrame);
rxTrainFrames = readall(trainFrames,"UseParallel",pctExists);
rxTrainFrames = cat(4, rxTrainFrames{:});
validFrames = transform(validDSTrans, @helperModClassReadFrame);
rxValidFrames = readall(validFrames,"UseParallel",pctExists);
rxValidFrames = cat(4, rxValidFrames{:});

% Read the training and validation labels into the memory
trainLabels = transform(trainDSTrans, @helperModClassReadLabel);
```

```
rxTrainLabels = readall(trainLabels,"UseParallel",pctExists);
validLabels = transform(validDSTrans, @helperModClassReadLabel);
rxValidLabels = readall(validLabels,"UseParallel",pctExists);
```

%本示例使用的 CNN 由六个卷积层和一个全连接层组成。除最后一个卷积层外，每个卷积层后面都有一个批量归一化层、修正线性单元

%(ReLU) 激活层和最大池化层。在最后一个卷积层中，最大池化层被一个平均池化层取代。输出层具有 softmax 激活。

```
modClassNet = helperModClassCNN(modulationTypes,sps,spf);
```

下来配置 [TrainingOptionsSGDM](#) 以使用小批量大小为 256 的 SGDM 求解器。将最大轮数设置为 12，因为更多轮数不会提供进一步的训练优势。

```
maxEpochs = 12;
miniBatchSize = 256;
options = helperModClassTrainingOptions(maxEpochs,miniBatchSize,...
    numel(rxTrainLabels),rxValidFrames,rxValidLabels);
```

```
if trainNow == true
    elapsedTime = seconds(toc);
    elapsedTime.Format = 'hh:mm:ss';
    fprintf('%s - Training the network\n', elapsedTime)
    trainedNet =
trainNetwork(rxTrainFrames,rxTrainLabels,modClassNet,options);
else
    load trainedModulationClassificationNetwork
end

elapsedTime = seconds(toc);
elapsedTime.Format = 'hh:mm:ss';
fprintf('%s - Classifying test frames\n', elapsedTime)
% Read the test frames into the memory
testFrames = transform(testDSTrans, @helperModClassReadFrame);
rxTestFrames = readall(testFrames,"UseParallel",pctExists);
rxTestFrames = cat(4, rxTestFrames{:});

% Read the test labels into the memory
testLabels = transform(testDSTrans, @helperModClassReadLabel);
```

```

rxTestLabels = readall(testLabels,"UseParallel",pctExists);
% 分类测试
rxTestPred = classify(trainedNet,rxTestFrames);
testAccuracy = mean(rxTestPred == rxTestLabels);
disp("Test accuracy: " + testAccuracy*100 + "%")

```

```

figure
cm = confusionchart(rxTestLabels, rxTestPred);
cm.Title = 'Confusion Matrix for Test Data';
cm.RowSummary = 'row-normalized';
cm.Parent.Position = [cm.Parent.Position(1:2) 740 424];

```

SDR 测试

```

radioPlatform = "ADALM-PLUTO";

switch radioPlatform
case "ADALM-PLUTO"
    if helperIsPlutoSDRInstalled() == true
        radios = findPlutoRadio();
        if length(radios) >= 2
            helperModClassSDRTest(radios);
        else
            disp('Selected radios not found. Skipping over-the-air test.')
        end
    end
case {"USRP B2xx","USRP X3xx","USRP N2xx"}
    if (helperIsUSRPInstalled() == true) && (helperIsPlutoSDRInstalled() ==
true)
        txRadio = findPlutoRadio();
        rxRadio = findsdru();
        switch radioPlatform
        case "USRP B2xx"
            idx = contains({rxRadio.Platform}, {'B200','B210'});
        case "USRP X3xx"
            idx = contains({rxRadio.Platform}, {'X300','X310'});
        case "USRP N2xx"
            idx = contains({rxRadio.Platform}, 'N200/N210/USRP2');
        end
        rxRadio = rxRadio(idx);
        if (length(txRadio) >= 1) && (length(rxRadio) >= 1)

```

```
        helperModClassSDRTest(rxRadio);  
    else  
        disp('Selected radios not found. Skipping over-the-air test.')    end  
end  
end  
end
```

附录 2：智能硬件部署

一、Cuda 代码生成环境配置（主机）

二、Jetson-nano 开发环境及 CUDA 环境搭建

三、Jeston cuda 代码部署与运行

四、无线信道下调制波形的收发与 jetson-nano 上的部署

Cuda 代码生成环境配置（主机）

Matlab 工具包安装：

深度学习所需工具包

Communication Toolbox

Deep Learning Toolbox

Parallel Computing Toolbox

USRP 所需工具包

DSP System Toolbox

Signal Processing Toolbox

Audio Toolbox

Computer Vision Toolbox

Communications Toolbox Support Package for USRP Radio

Jetson-nano 所需工具包

Wavelet Toolbox

Image Processing Toolbox

MATLAB Coder

GPU Coder

MATLAB Coder Support Package for NVIDIA Jetson and NVIDIA Drive Platforms